

# 用于 DINO-WM CEM 规划的 RankSafe 前缀缓存

方法名称: RankSafe Exact-Prefix Cache (RankSafe-EPC)

## 研究动机

DINO-WM 的 test-time CEM planner 会从同一段观测历史出发, 评估多条可能的 action sequence。这些候选共享 action-conditioned dynamics 之前的 visual/history 计算, 但直接实现时会为每条分支重复计算。因此, 问题不只是 inference 速度: 任何复用规则都必须保持候选 cost 的排序不变, 因为 planner 根据这个排序决定首个 action。即使单步 prediction 看起来接近, 缓存若悄悄改变了排序, closed-loop 行为也可能随之改变。

$C^3$ ache 和 X-Cache 根据 residual similarity 或 action-aware fingerprint 决定何时复用计算, WorldCache 则利用 curvature 和 drift 在 rollout fidelity 与速度之间取舍。这些工作说明复用计算是有用的, 但它们的 validity signal 没有精确说明: forward graph 中哪些计算与候选 action 无关。RankSafe-EPC 在 DINO-WM Wall planner 中处理这个缺口: 找出与 action 无关的 prefix, 保持 suffix 不变, 并直接检查 candidate ordering 和首个 selected action 是否保持不变。

近来的 cache system (如  $C^3$ ache、X-Cache 和 WorldCache) 让复用更实用, 但还没有建立 planner selection invariance。现有的 DINO-WM Wall checkpoint 和 local trajectories 提供了固定且可复现的环境, 可以在不 retraining 的情况下测量 graph instrumentation、exact-prefix reuse、native latency 和 first-action agreement。一个受限的实现和匹配评估可以在 2-4 张 A100 上完成。因此, 这个缺失的 contract 现在可以检验; 不过, 在取得 native end-to-end 和 closed-loop evidence 以前, 结论仍是 provisional。

已有工作停在这里有结构性原因。 $C^3$ ache 在平滑的连续 WAM chunk 之间复用 residual, 但没有分析 planner graph 中的 action independence。X-Cache 用近似的 structure- and action-aware fingerprint gate, 而没有证明 CEM 候选之间 planner-relevant prefix 相等并保持 selected action。WorldCache 针对 rollout content fidelity, 而不是 sampling-based planner 用来选择 action 的候选排序。EfficientVLA 通过 pruning、token selection 和 feature caching 改变所表示的计算, 并没有在 action-dependency boundary 处隔离 planner-preserving reuse。DINO-WM 本身直接计算每条 counterfactual branch, 也没有提供与 candidate-cost 和 selected-action invariance 绑定的 reusable prefix 或 cache-validity test。

如果这个缺口被补上, shared-prefix reuse 就会成为一种 planner-level operation, 并有明确条件来保持 CEM candidate ordering 和 selected first action, 而不再只是下游影响未知的 similarity heuristic。它也会给  $C^3$ ache 和 X-Cache 这样的 system 一个具体的 diagnostic boundary, 用来区分 safe exact reuse 与 approximate cross-chunk reuse, 同时共同测量 native latency 和 closed-loop agreement。

## 方法

### M1\_mechanism

找出精确的 action-independent boundary, 复用 shared prefix, 同时保留每个 action-conditioned suffix。

1. 在 evaluation mode 下加载 DINO-WM Wall checkpoint。把每个 action vector 作为显式输入, 并按从 observation/history encoder 到 rollout head 的 topological order 列出 tensor boundary。对当前 Cross-Entropy Method (CEM) population 的每个 action vector, 在每个 boundary 计算 automatic-differentiation Jacobian; 只要某个 action component 的导数非零, 就把对应 boundary element 标为依赖, 并对所有候选逐元素合并 mask。选择合并 mask 全为零的最晚 boundary, 保存其 module name、tensor shape、data type、device 和 binary mask; 如果不存在, 就记录 'no\_certified\_boundary', 本轮走 full graph。observation/history 或 CEM population 改变时重新构造 mask。二值 mask 标记边界  $b$  处依赖候选 action 的图节点。

$$M_b = \mathbf{1}\{\partial h_b / \partial a \neq 0\} \quad (1)$$

为什么: 重复发生的计算是 shared prefix, 因此显式 dependency mask 能提供判断这个 prefix 是否可复用的精确条件。

2. 对于每个 observation/history, 在 certified boundary 只运行一次 shared prefix, 并保持与 full recomputation 相同的 evaluation mode、data type、device、model weights 和 preprocessing。把 boundary tensor 与 observation/history identifier、checkpoint revision、boundary identifier、shape、data type 和 device 一起存入 cache key。命中 cache 时, 将该 tensor 以只读形式提供给每个 candidate-specific suffix; 每次 suffix 调用配对对自己的 candidate action, 且不改变 suffix 参数。任一 key 字段变化就使 entry 失效, 并拒绝 in-place write, 避免某个 suffix 改变其他候选看到的 representation。当边界与 action 无关时, 一个 cached prefix 表示可供全部候选  $j$  使用。

$$M_b = 0 \implies h_b^{(j)} = h_b^{\text{cache}} \quad \forall j \in \{1, \dots, K\} \quad (2)$$

为什么: Exact reuse 去掉重复计算, 同时保持决定 counterfactual outcome 的 action-conditioned 部分不变。

## M2\_validation

保护 *candidate evaluation*, 并检验 *planner invariance*、下游 *decision* 和 *native latency*。

3. 运行每个 action-conditioned suffix, 保存 predicted observation sequence 和每个 horizon step 的 loss。使用 planner 提供的 rollout loss ‘ell’, 将每个 predicted observation 与对应的 reference observation 比较, 并把整个 horizon 的 loss 相加, 得到每个候选的 scalar objective, 同时保留候选顺序。【作者需决定: 明确 ‘ell’ 的具体形式、reference observation 的来源、归一化方式及 horizon 约定; 这些决定会影响候选排序。】将固定 population order 中的第一个 candidate 作为 deterministic canary: 用 uncached full graph 再算一次, 把 boundary tensor 和最终 objective 与 cached 结果做 bitwise tensor equality (‘torch.equal’); 任一 mismatch 就使当前 cache entry 失效, 并让所有 candidate 走 full graph。对独立 negative control, 把 ‘ $M_b$ ’ 展平, 用由 episode、observation/history identifier 和 CEM iteration 派生的 seed 做 uniform permutation, 保持 1 的数量; 若 permutation 未改变 mask 就重抽 seed, 关闭 fallback, 运行同一 suffix 和 objective pipeline, 并记录 seed、corrupted mask、per-candidate objectives 和 mismatch。CEM 根据共享 prefix 与候选专属 suffix 得到的 rollout cost 对候选排序。

$$J_j = \sum_{\tau=1}^H \ell(\hat{o}_{j,\tau}, o_{t+\tau}), \quad \pi = \text{argsort}(J_{1:K}) \quad (3)$$

为什么: 如果 gate 被它本应保护的结果污染, 它可能掩盖错误。canary 和匹配的 corrupted-mask control 可以暴露这种 failure mode。

4. 在相同 CEM settings 下, 使用相同的 candidate population、random seed、initial history、checkpoint、preprocessing、rollout horizon 和 action bounds, 分别运行 full recomputation、有效 mask 的 cached path (RankSafe) 以及 randomized-mask control。对每个 planner population, 按 scalar objective 做 stable sort (objective 相同则按 candidate index); 只有完整排序不同才记为 rank disagreement=1, 只有最低 objective candidate 选出的 first action vectors 逐元素相等才记为 first-action agreement=1。对所有 population 分别取 binary 指标的 arithmetic mean, 并保留逐 population 记录。从相同 initial state 和 seed 开始执行 closed-loop episodes, 再用现有 task evaluator 产生 success outcome。【作者需决定: 明确 task success 的判定条件、episode 终止条件和 outcome 汇总规则, 否则 closed-loop success 无法复现。】将 native end-to-end latency 的计时边界设为 planner call 开始到返回 first action; 所有 path 使用相同 warm-up 和 device synchronization policy, 保留每次调用时长, 并报告 arithmetic mean 和 median。【作者需决定: 确认计时是否包含 input preprocessing、synchronization 和 environment interaction; 该选择定义 native latency。】机制检查记录排序是否不一致, 并要求 cached 与 full recomputation 的首 action 相同。

$$\Delta_{\text{rank}} = \mathbf{1}[\pi_{\text{cache}} \neq \pi_{\text{full}}], \quad a_{\text{cache}}^* = a_{\text{full}}^* \quad (4)$$

为什么: 这个 proposal 关注的是 planner 的 selection quantity, 因此只有速度还不够; 还必须保持排序和 action agreement。